3-1

과정1 리뷰 & 과정2 목표

학습 목차

- **과정 1 리뷰**
  - LLM, Agent, LangChain
  - RAG (Streamlit)
  - 복잡한 Agent 표현을 위한 LangGraph
- **과정 2 목표**
  - MCP와 A2A
  - LangGraph 심화
  - Harness Engineering
  - Observability

© Scopelabs. All Rights Reserved

과정 1 리뷰

LLM, Agent, LangChain

LLM은 무엇이며, 어떻게 컴퓨터를 사용하는 Agent가 되었는가.

- LLM: 방대한 양의 언어 데이터로 부터, "이 다음엔 어떤 말이 나올까?"를 예측하는 모델
- Function(Tool) Calling: LLM의 출력을 해석해서, 컴퓨터의 기능을 실행하는 행위
- Agent: 함수 호출과 그 결과를 검토하는 과정을 반복, 복잡한 작업을 처리하도록 구성된 LLM 파이프라인
- LangChain(LangGraph): LLM을 활용한 워크플로우를 만드는데 사용되는 라이브러리들

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "너는 초보자를 위한 AI 튜터야."),
    ("human", "{topic}을 초급자에게 3문장으로 설명해줘."),
])

parser = StrOutputParser()
chain = prompt | model | parser

answer = chain.invoke({"topic": "LLM"})
```

RAG (Streamlit)

LLM의 응답의 품질을 올리기 위한 기법 중 하나.

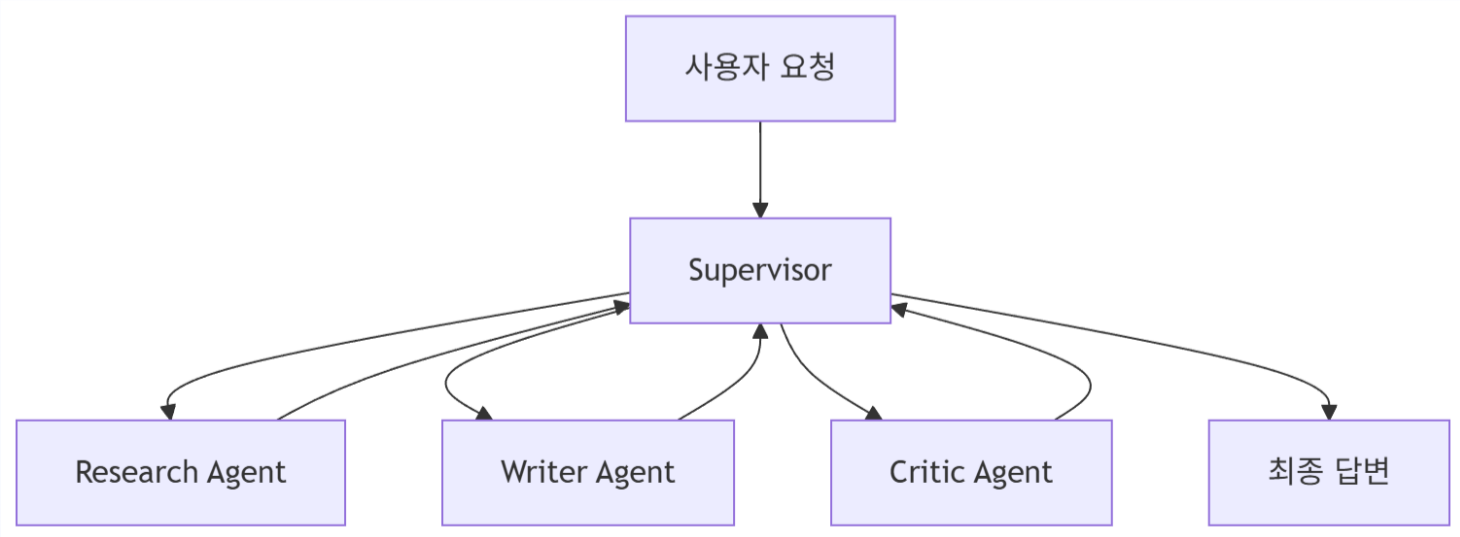
- 문서를 쪼개고 의미의 유사성을 비교할 수 잇는 형태로 변형
- 사용자 질문과 가장 연관성 높은 문서를 검색
- LLM이 답변을 생성할 때 문서 내용을 더해 답변 품질 향상



복잡한 Agent 표현을 위한 LangGraph

LangGraph는 Agent 실행 흐름을 그래프로 표현해 복잡한 분기와 반복을 다룰 수 있게 한다.

- State를 중심으로 실행 맥락 관리
- Node와 Edge로 Agent 흐름 구성
- Conditional Edge로 조건부 분기 처리
- ToolNode, Human-in-the-loop 같은 Agent 패턴 표현



과정 2 목표

MCP와 A2A

과정 2에서는 Agent 시스템이 외부 도구와 다른 Agent를 표준 방식으로 연결하는 흐름을 다룬다.

- MCP: Agent와 Tool 또는 데이터 소스 연결
- A2A: Agent와 Agent 사이의 작업 위임

LangGraph 심화

LangGraph 심화에서는 Agent 실행 흐름을 더 안정적이고 확장 가능한 구조로 다듬는다.

- Subgraph로 그래프 모듈화
- Persistence로 상태 저장과 재개
- Parallel Execution으로 병렬 처리
- Memory Store로 장기 기억 관리

Harness Engineering

Harness Engineering은 Agent가 안정적으로 일할 수 있도록 실행 환경과 맥락을 설계하는 관점이다.

- 필요한 컨텍스트를 어떻게 제공할지 설계
- 도구 사용 조건과 실패 복구 전략 정의
- Skill, 지침, 파일 구조를 Agent가 읽기 쉽게 정리

Observability

Observability는 Agent가 실제로 어떻게 실행됐는지 추적하고 평가하는 역량이다.

- Trace로 모델 호출, 도구 실행, 체인 흐름 확인
- Score로 응답 품질을 수치화
- Langfuse로 실행 로그와 평가 결과 관리
- RAG와 Agent 응답을 지속적으로 개선